EXPERMENT: To play Tic-Tac-Toe.

Aim

To design and implement a Tic-Tac-Toe game using Python that allows a human player to play against the computer.

Objectives

1. To understand game development using Python.
2. To represent a Tic-Tac-Toe board programmatically.
3. To accept player input and validate moves.
4. To identify winning and draw conditions.
5. To implement computer-based moves.

Algorithm

1. Create a 3 × 3 Tic-Tac-Toe board.
2. Assign X to the human player and O to the computer.
3. Display the board.
4. Ask the player to select a position.
5. Check whether the selected position is empty.
6. Place X in the selected position.
7. Check for a winning combination.
8. If the player wins, display the result and stop.
9. Otherwise, allow the computer to make a move.
10. Check whether the computer wins or the game is a draw.
11. Repeat until a player wins or the board is full.

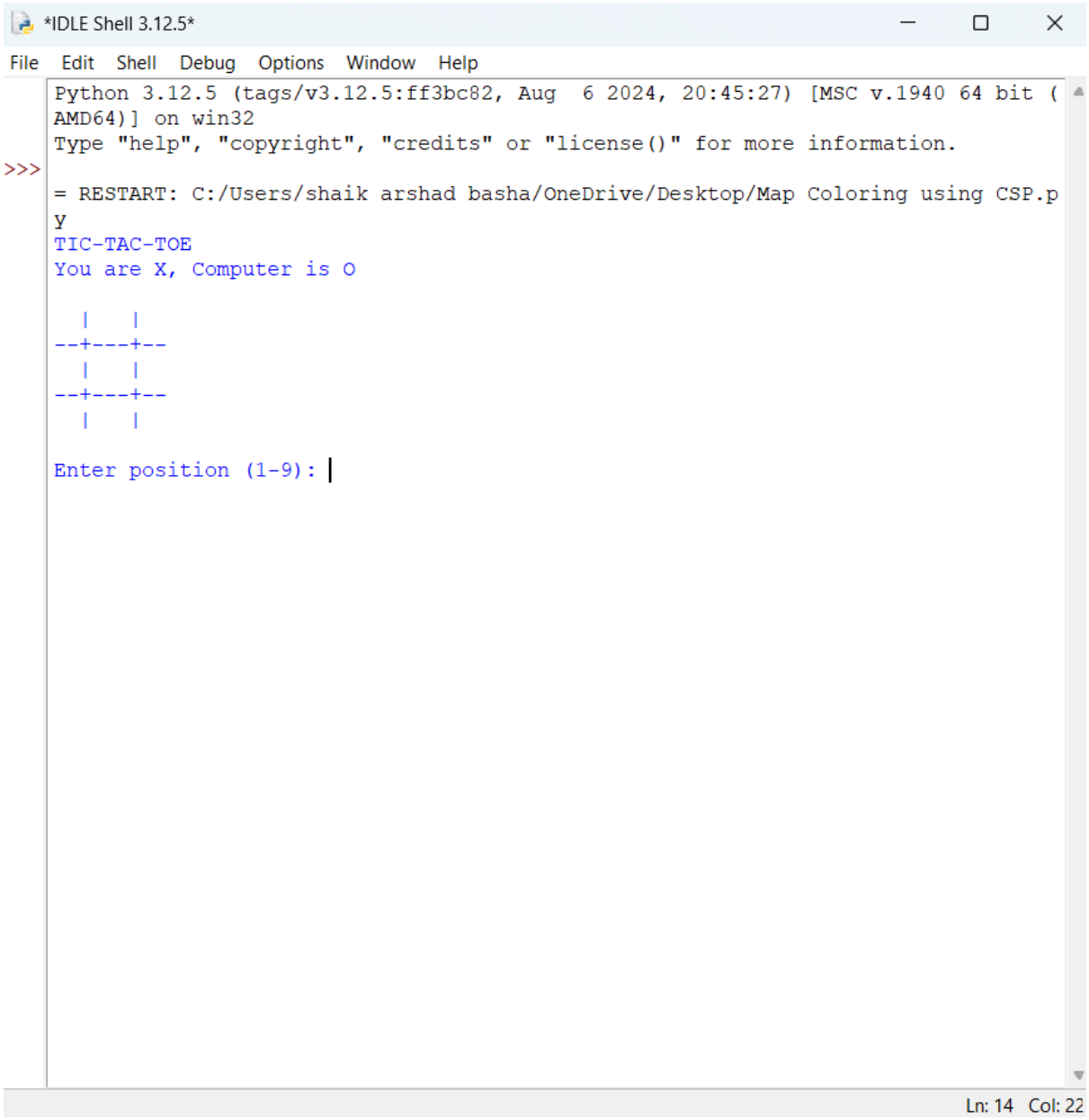
CODE

```
# Tic-Tac-Toe Game

board = [' ' for _ in range(9)]

def display_board():
    print()
    print(board[0], '|', board[1], '|', board[2])
    print('--+---+--')
    print(board[3], '|', board[4], '|', board[5])
    print('--+---+--')
```

```
    print(board[6], '|', board[7], '|', board[8])
    print()
def check_winner(player):
    winning_positions = [
        (0, 1, 2),
        (3, 4, 5),
        (6, 7, 8),
        (0, 3, 6),
        (1, 4, 7),
        (2, 5, 8),
        (0, 4, 8),
        (2, 4, 6)
    ]
    for a, b, c in winning_positions:
        if board[a] == board[b] == board[c] == player:
            return True
    return False
def is_full():
    return ' ' not in board
print("TIC-TAC-TOE")
print("You are X, Computer is O")
while True:
    display_board()
    # Player move
    try:
        position = int(input("Enter position (1-9): ")) - 1
    except ValueError:
        print("Enter a number from 1 to 9.")
        continue
    if position < 0 or position > 8 or board[position] != ' ':
        print("Invalid move. Try again.")
```

```
    continue
board[position] = 'X'
if check_winner('X'):
    display_board()
    print("You Win!")
    break
if is_full():
    display_board()
    print("Draw!")
    break
# Computer move
for i in range(9):
    if board[i] == ' ':
        board[i] = 'O'
        break
if check_winner('O'):
    display_board()
    print("Computer Wins!")
    break
if is_full():
    display_board()
    print("Draw!")
    break
```

OUTPUT



Result

The Tic-Tac-Toe game was successfully implemented in Python. The program accepts user moves, generates computer moves, and correctly identifies win, loss, and draw conditions.

Conclusion

The experiment demonstrates how Python can be used to develop a simple interactive game using arrays, functions, conditional statements, and loops.